

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Математико-механический факультет

Сагорен Кирилл Давидович

Оптимизация Лабораторной работы 1 с помощью добавления параллельности

Лабораторная работа

студента 1 курса дневного отделения

Санкт-Петербург

2025

Цель работы: ускорить выполнение операций обработки BMP-изображений (поворот и применение гауссова фильтра) за счет внедрения параллельных вычислений с использованием OpenMP.

1. Основные изменения

В код были добавлены директивы OpenMP для параллелизации трех ключевых операций:

1. rotateRight() – поворот изображения вправо
2. rotateLeft() – поворот изображения влево
3. applyGaussianFilter() – применение гауссова размытия

Изменения в коде:

RotateClockwise90():

```
void ClassBMP::RotateClockwise90()
{
    int old_row_size = (image_info.image_width * (image_info.bits_per_pixel / 8) + 3) & ~3;
    int new_row_size = (image_info.image_height * (image_info.bits_per_pixel / 8) + 3) & ~3;

    std::vector<uint8_t> rotated_data(new_row_size * image_info.image_width);

    #pragma omp parallel for
    for (int y = 0; y < image_info.image_height; ++y)
    {
        for (int x = 0; x < image_info.image_width; ++x)
        {
            int old_index = y * old_row_size + x * (image_info.bits_per_pixel / 8);
            int new_index = (image_info.image_width - x - 1) * new_row_size + y * (image_info.bits_per_pixel / 8);

            for (int channel = 0; channel < (image_info.bits_per_pixel / 8); ++channel)
            {
                rotated_data[new_index + channel] = pixel_data[old_index + channel];
            }
        }
    }

    std::swap(image_info.image_width, image_info.image_height);
    image_info.raw_image_size = rotated_data.size();
    file_header.total_file_size = file_header.pixel_data_offset + rotated_data.size();

    pixel_data = std::move(rotated_data);
}
```

Рис1. Изменение RotateClockwise90():

RotateCounterClockwise90():

```

void ClassBMP::RotateCounterClockwise90()
{
    int old_row_size = (image_info.image_width * (image_info.bits_per_pixel / 8) + 3) & ~3;
    int new_row_size = (image_info.image_height * (image_info.bits_per_pixel / 8) + 3) & ~3;

    std::vector<uint8_t> rotated_data(new_row_size * image_info.image_width);

    #pragma omp parallel for
    for (int y = 0; y < image_info.image_height; ++y)
    {
        for (int x = 0; x < image_info.image_width; ++x)
        {
            int old_index = y * old_row_size + x * (image_info.bits_per_pixel / 8);
            int new_index = x * new_row_size + (image_info.image_height - y - 1) * (image_info.bits_per_pixel / 8);

            for (int channel = 0; channel < (image_info.bits_per_pixel / 8); ++channel)
            {
                rotated_data[new_index + channel] = pixel_data[old_index + channel];
            }
        }
    }

    std::swap(image_info.image_width, image_info.image_height);
    image_info.raw_image_size = rotated_data.size();
    file_header.total_file_size = file_header.pixel_data_offset + rotated_data.size();

    pixel_data = std::move(rotated_data);
}

```

Рис2. Изменение RotateCounterClockwise90():

ApplyGaussianFilter():

```

void ClassBMP::ApplyGaussianBlur()
{
    const float kernel[5][5] =
    {
        {1/256.0f, 4/256.0f, 6/256.0f, 4/256.0f, 1/256.0f},
        {4/256.0f, 16/256.0f, 24/256.0f, 16/256.0f, 4/256.0f},
        {6/256.0f, 24/256.0f, 36/256.0f, 24/256.0f, 6/256.0f},
        {4/256.0f, 16/256.0f, 24/256.0f, 16/256.0f, 4/256.0f},
        {1/256.0f, 4/256.0f, 6/256.0f, 4/256.0f, 1/256.0f}
    };

    std::vector<uint8_t> blurred_data(pixel_data.size());
    int channels = image_info.bits_per_pixel / 8;

    #pragma omp parallel for
    for (int y = 2; y < image_info.image_height - 2; y++)
    {
        for (int x = 2; x < image_info.image_width - 2; x++)
        {
            for (int c = 0; c < channels; c++)
            {
                float color = 0.0f;

                for (int ky = -2; ky <= 2; ky++)
                {
                    for (int kx = -2; kx <= 2; kx++)
                    {
                        int pixel_x = x + kx;
                        int pixel_y = y + ky;
                        color += pixel_data[(pixel_y * image_info.image_width + pixel_x) * channels + c] * kernel[ky + 2][kx + 2];
                    }
                }

                blurred_data[(y * image_info.image_width + x) * channels + c] = static_cast<uint8_t>(color);
            }
        }
    }

    pixel_data = blurred_data;
}

```

Рис3. Изменения ApplyGaussianFilter()

2. Изменения в Makefile:

В Makefile добавлен флаг -fopenmp для поддержки OpenMP.

Измерение результатов

С помощью <chrono> было измерено время выполнения программы до и после оптимизации.

3. Результаты измерения:

После оптимизации время работы моей программы снизилось в два раза (приблизительно)

До применения параллельности:

```
Rotating image clockwise by 90 degrees...
Right rotation time: 14 ms
Rotating image counter-clockwise by 90 degrees...
Left rotation time: 5 ms
Applying Gaussian Blur to the image...
Writing filtering time: 234 ms
All time taken: 817 ms
Image processing completed successfully!
```

Рис 6.Время работы до оптимизации

После применения:

```
Rotating image clockwise by 90 degrees...
Right rotation time: 4 ms
Rotating image counter-clockwise by 90 degrees...
Left rotation time: 4 ms
Applying Gaussian Blur to the image...
Writing filtering time: 164 ms
All time taken: 436 ms
Image processing completed successfully!
```

Рис 7. Время работы после оптимизации

